

Simulation de Cellules 2D

Propagation d'une maladie sur une grille 2D

Rapport de projet



Sain



Infecté



Guéri



Décédé

Projet P.G.L. (Projet de Génie Logiciel)

Filière ING1 Génie Informatique, CYTech, 2025-2026

Maikel Moawad • Ryan Boudraa

Dépôt : github.com/Maikimaik-18/Simulation-de-cellules-2D

Juin 2026

1. Présentation de l'équipe et du sujet

1.1 L'équipe

Ce projet a été réalisé en binôme par **Maikel Moawad** et **Ryan Boudraa**, étudiants en première année du cycle ingénieur Génie Informatique à CYTech. Le cahier des charges recommande des groupes de cinq personnes ; notre équipe a fonctionné à deux, sur dérogation validée par le directeur de projet. Cette taille réduite a augmenté la charge individuelle, mais elle a aussi simplifié la coordination et permis à chacun de maîtriser l'intégralité de sa moitié du code, ce qui est un atout pour la soutenance.

1.2 Le sujet

Le sujet consiste à concevoir une application graphique simulant le comportement de cellules dans un plan 2D. Les cellules évoluent selon trois facteurs : le temps (naissance, mort, déplacement), l'environnement (cellules voisines, interactions) et les actions de l'utilisateur, qui peut modifier les cellules en direct et observer les conséquences.

Conformément à notre périmètre Éthique et Design, nous avons retenu comme thème la **propagation d'une maladie** au sein d'une population. Ce thème s'appuie sur le modèle épidémiologique SIR, auquel nous avons ajouté un état de décès. Chaque case de la grille est soit vide, soit occupée par une personne se trouvant dans l'un des quatre états suivants :

État	Couleur	Signification
SUSCEPTIBLE	Bleu	Personne saine, peut être contaminée
INFECTED	Rouge	Personne malade, contamine ses voisins
RECOVERED	Vert	Personne guérie, immunisée
DECEASED	Gris	Personne morte, ne participe plus à la simulation

À chaque tour de simulation, chaque personne vieillit (son compteur de jours d'infection augmente si elle est malade), peut guérir ou mourir selon des probabilités, se déplace éventuellement vers une case voisine vide, puis contamine ses voisins sains si elle est infectée. Les probabilités de transmission, de guérison, de mortalité et de mobilité sont réglables en temps réel depuis l'interface.

2. Périmètre fonctionnel

L'application couvre l'ensemble des fonctionnalités exigées par le cahier des charges. Le tableau ci-dessous récapitule cette couverture.

Exigence du cahier des charges	Réalisation
Grille configurable, topologie bornée ou torique	Dimensions réglables ; mode torique géré par Math.floorMod
Caractéristiques individuelles des cellules	État, position, jours d'infection, facteur d'immunité, statut vaccinal
Création, modification, suppression de cellules	Clic individuel, pinceau au glisser, remplissage aléatoire de masse
Réglage de la vitesse, pas à pas, pause, reprise	Curseur de vitesse, boutons Play / Pause / Step / Reset
Règles d'évolution (déplacement, propagation, mort)	Méthodes update, move, spread de la classe Person

Exigence du cahier des charges	Réalisation
Plusieurs modes d'interaction (voisinage)	4 stratégies : orthogonal, Moore, euclidien, Manhattan
Probabilités globales ou individuelles	Probabilités globales réglables et immunité propre à chaque personne
Statistiques en direct et graphiques	Compteurs par état, courbes temps réel, statistiques de propriétés
Sauvegarde et restauration au format binaire	Sérialisation Java via SaveService (ObjectOutputStream)
Version ligne de commande et version JavaFX	CliRunner (environ 13 commandes) et interface graphique JavaFX

Au delà du socle exigé, trois extensions ont été ajoutées : l'immunité individuelle (chaque personne reçoit un facteur d'immunité tiré autour d'une moyenne réglable, avec une dispersion réglable), la vaccination (pinceau dédié et bouton de vaccination d'un pourcentage de la population, avec une efficacité réglable), et la peinture au glisser de la souris.

3. Conception et architecture

Le soin apporté à la conception en amont, demandé par le cahier des charges, s'est traduit par une architecture modulaire en six packages, où la logique métier est strictement séparée des interfaces. Aucune classe métier (model, neighborhood, stats, io) n'importe JavaFX : seule la couche app en dépend. Cette séparation est ce qui permet d'avoir deux versions de l'application, en ligne de commande et graphique, partageant exactement le même moteur.

Package	Rôle
model	Cœur de la simulation : Grid, Person, Position, DiseaseState, SimulationSettings, SimulationEngine
neighborhood	Les 4 stratégies de voisinage interchangeables
stats	Calcul et historique des statistiques (pattern Observer)
io	Sauvegarde et chargement binaire (SaveService)
cli	Version en ligne de commande (CliRunner)
app	Interface graphique JavaFX (Main, MainController)

3.1 Patrons de conception

Deux patrons de conception structurent le projet et relient ses deux moitiés par de simples interfaces :

Strategy : l'interface NeighborhoodStrategy définit le contrat d'un mode de voisinage. Ses quatre implémentations (orthogonal à 4 cases, Moore à 8 cases, euclidien par rayon, Manhattan par distance) sont interchangeables à chaud, sans modifier le moteur.

Observer : l'interface SimulationListener permet au moteur de notifier des observateurs à chaque tour, sans connaître leur nature. Le service de statistiques et le contrôleur graphique s'y abonnent.

S'y ajoutent un découpage de type **Service** (StatisticsService et SaveService isolent les responsabilités transverses) et une organisation globale de type **MVC** entre le modèle, la vue FXML et le contrôleur.

3.2 Diagramme de classes

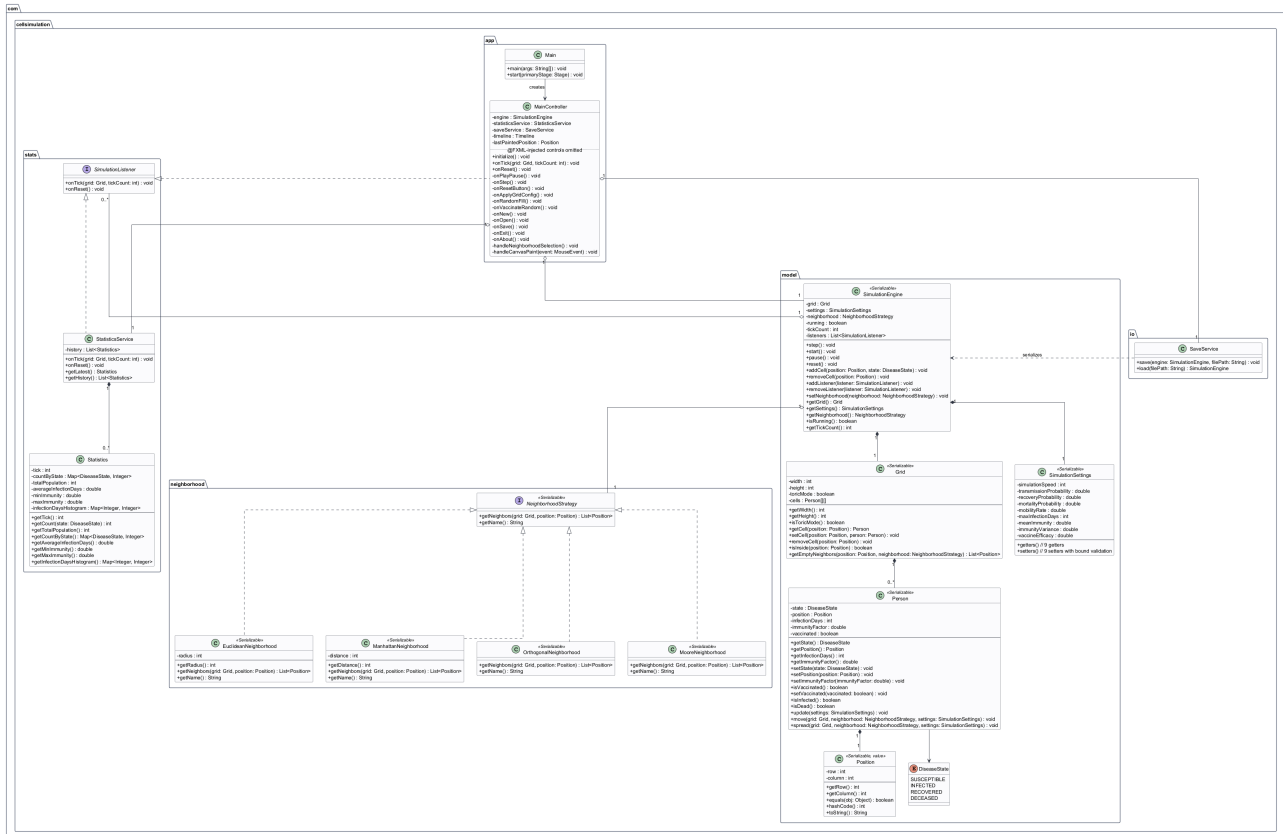


Figure 1. Diagramme de classes de l'application.

3.3 Déroulement d'un tour de simulation

Le cœur de la boucle est la méthode `step()` de `SimulationEngine`. À chaque tour, le moteur parcourt toutes les positions de la grille et, pour chaque personne, applique dans l'ordre `update` (vieillesse, guérison ou mortalité), `move` (déplacement) puis `spread` (contamination). Il notifie ensuite ses observateurs, qui mettent à jour les statistiques et l'affichage.

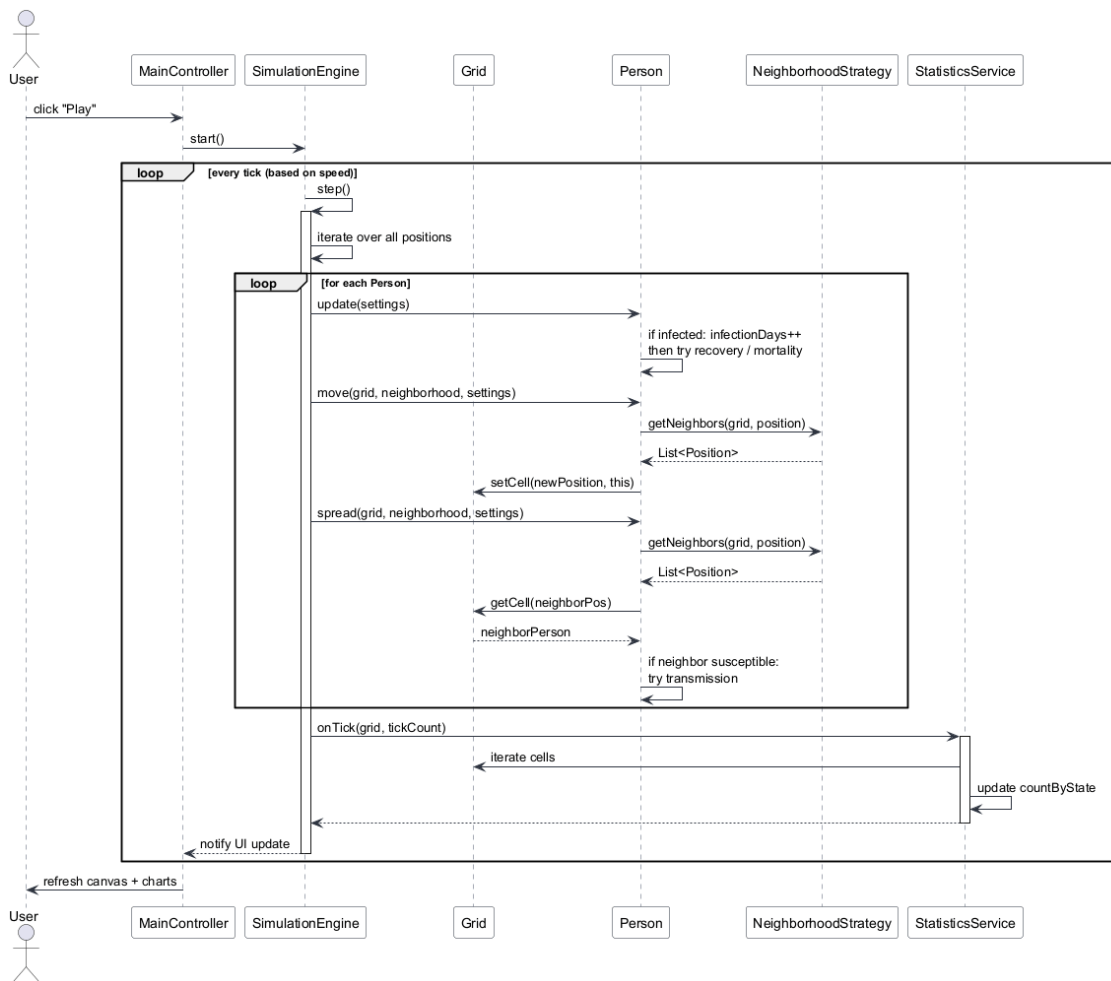


Figure 2. Diagramme de séquence d'un tour de simulation.

4. Cas d'utilisation

Du point de vue de l'utilisateur, l'application s'organise autour de cinq familles d'actions : le contrôle de la simulation, sa configuration, la gestion des cellules, la consultation des statistiques et la persistance.

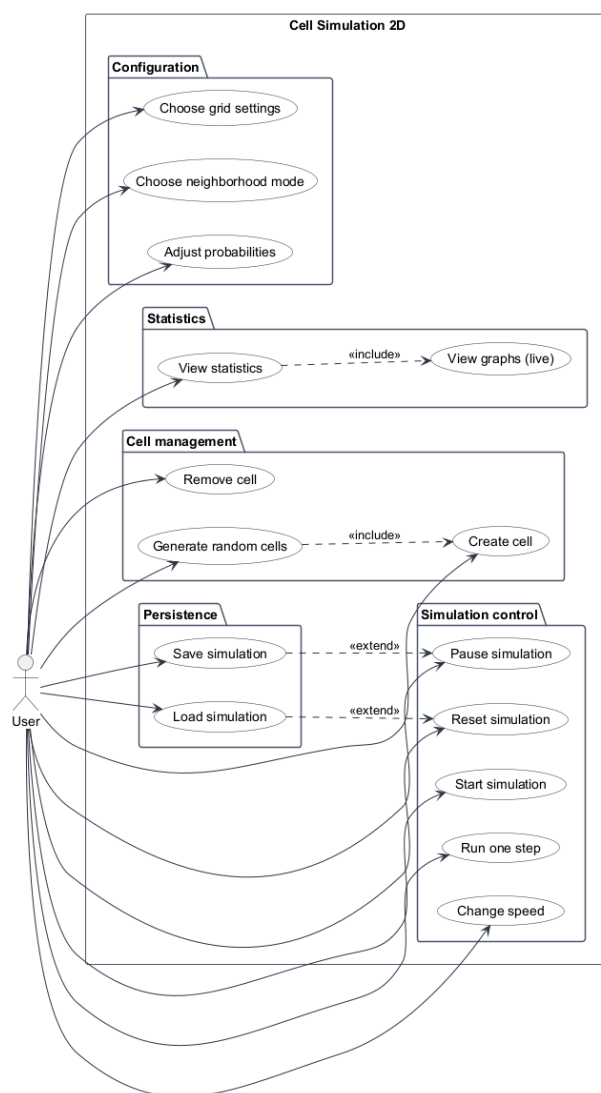


Figure 3. Diagramme des cas d'utilisation.

Cas d'utilisation	Description
Contrôler la simulation	Démarrer, mettre en pause, avancer d'un pas, réinitialiser, régler la vitesse
Configurer	Choisir les dimensions et la topologie de la grille, le mode de voisinage, ajuster les probabilités
Gérer les cellules	Créer une personne, en supprimer, générer une population aléatoire
Consulter les statistiques	Lire les compteurs par état et suivre les courbes en direct
Sauvegarder et charger	Enregistrer l'état complet dans un fichier binaire, puis le restaurer

5. Organisation de l'équipe et flux de travail

Nous avons découpé le projet en deux moitiés cohérentes qui ne communiquent que par des interfaces, de sorte que chacun maîtrise entièrement sa partie.

Membre	Périmètre	Responsabilités
Maikel Moawad	Le moteur : packages model et neighborhood	Algorithme de la maladie, grille bornée et torique, pattern Strategy, moteur de simulation

Membre	Périmètre	Responsabilités
Ryan Boudraa	Services et interfaces : packages stats, io, cli, app	Pattern Observer, statistiques, sérialisation, version ligne de commande, interface JavaFX

La règle de dépendance est unidirectionnelle : la partie services et interfaces utilise le moteur, jamais l'inverse. Les deux moitiés se rejoignent en exactement trois points de jonction :

1. **L'API publique de SimulationEngine**, que les interfaces appellent (step, addCell, setNeighborhood, addListener).
2. **L'interface NeighborhoodStrategy** (pattern Strategy) : le contrat est côté moteur, l'instanciation et le choix côté interface.
3. **L'interface SimulationListener** (pattern Observer) : le contrat est côté services, et c'est le moteur qui l'appelle. La symétrie de ces deux patrons, contrat d'un côté et utilisation de l'autre, illustre la séparation des responsabilités.

5.1 Outils et méthode

Besoin	Outil
Gestion de versions	Dépôt Git centralisé sur GitHub
Communication	WhatsApp et appels téléphoniques
Suivi des tâches	Page HTML de suivi partagée
Convention de code	Google Java Style (formatage automatique)

Le développement a suivi une progression par phases successives (du socle métier vers les interfaces, puis les extensions et la finition). Chaque phase a fait l'objet de commits ciblés et documentés, en respectant l'exigence d'au moins un commit par jour. Tout le code, les variables, les commentaires et la JavaDoc sont rédigés en anglais, comme l'impose le cahier des charges.

6. Problèmes rencontrés et solutions

6.1 Difficultés d'organisation

Équipe réduite à deux. Travailler à deux plutôt qu'à cinq a fortement augmenté la charge par personne. Nous y avons répondu en cadrant strictement le périmètre sur le cahier des charges, sans développer de fonctionnalité superflue, et en privilégiant une application stable plutôt qu'une accumulation de fonctions fragiles, conformément à la consigne du sujet.

Coordination à distance. Une grande partie du travail s'est faite à distance. Le découpage net en deux moitiés reliées par interfaces, combiné à un dépôt Git commun et à une page de suivi des tâches, nous a permis de travailler en parallèle sans nous bloquer mutuellement.

6.2 Difficultés techniques

Sérialisation des stratégies de voisinage. Lors des premiers tests de sauvegarde, une `NotSerializableException` était levée sur les classes de voisinage : le moteur sérialisait son champ de stratégie, mais celles-ci n'étaient pas sérialisables. Nous avons fait hériter l'interface `NeighborhoodStrategy` de `Serializable` et ajouté un `serialVersionUID` aux quatre implémentations, qui ne portent que des données primitives. La sauvegarde et le rechargement sont alors devenus fiables.

Affichage de la version ligne de commande sous Windows. Un caractère tiret cadratin utilisé dans la bannière de la CLI s'affichait incorrectement dans la console Windows (page de codes cp850). Nous l'avons remplacé par un tiret simple pour garantir un rendu correct sur toutes les plateformes.

Voisinage euclidien. Plutôt que de calculer une racine carrée pour chaque case candidate, le voisinage euclidien compare des carrés de distances (en entiers longs pour éviter tout débordement). Le résultat est mathématiquement équivalent, plus rapide et sans erreur d'arrondi flottant.

Cohérence des diagrammes UML. Au fil du développement, le code a évolué plus vite que les diagrammes. Une phase de finition dédiée a permis de réaligner le diagramme de classes et le diagramme de séquence sur le code réellement livré.

Choix assumé sur les tests. Le cahier des charges exige une documentation Javadoc et une gestion rigoureuse des erreurs, mais n'impose pas de tests unitaires. Nous avons fait le choix de ne pas écrire de tests JUnit afin de concentrer notre effort sur la stabilité et sur le périmètre demandé.

7. Résultats

Le projet atteint les objectifs du cahier des charges. L'application se compile et se lance en ligne de commande (mvn clean compile aboutit à un BUILD SUCCESS sur les sources du projet), ce qui correspond au mode de démonstration prévu le jour de la soutenance : récupération du dépôt, compilation, puis lancement sans environnement de développement.

Les deux versions demandées sont opérationnelles et partagent le même moteur métier : la version JavaFX, évaluée, et la version en ligne de commande, qui permet de présenter toutes les fonctionnalités du modèle indépendamment de l'interface graphique. La gestion des erreurs a été soignée aux points d'entrée (saisies utilisateur et entrées-sorties fichier) afin que l'application ne s'interrompe jamais de manière intempestive, exigence fortement pénalisante dans la notation.

Enfin, une documentation Javadoc générée est présente dans le dépôt, et les trois diagrammes (classes, cas d'utilisation, séquence) accompagnent le code livré.

8. Conclusion

Ce projet nous a permis de mettre en pratique, sur une application complète et fonctionnelle, la conception orientée objet, les patrons de conception Strategy et Observer, la sérialisation, et le développement d'une interface graphique JavaFX. La contrainte d'une équipe réduite à deux personnes nous a appris à cadrer notre périmètre et à nous répartir le travail de façon claire, chacun restant maître de sa moitié du code. Le découpage en deux parties reliées par des interfaces, au delà de son intérêt pédagogique, garantit que nous savons, pour chaque fonctionnalité, situer le fichier concerné, expliquer son rôle et montrer où il est utilisé, ce qui sera précieux lors de la soutenance.